

HOMEWORK 1

Kavya Saranya K V

2025115152

Q1. Scan two matrices and find their sum.

Code:

```
#include <stdio.h>

int main() {
    int r, c, i, j;

    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);

    int x[r][c], y[r][c], z[r][c];

    printf("Enter elements of matrix A:\n");
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &x[i][j]);

    printf("Enter elements of matrix B:\n");
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &y[i][j]);

    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            z[i][j] = x[i][j] + y[i][j];

    printf("Sum matrix:\n");
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++)
            printf("%d ", z[i][j]);
        printf("\n");
    }

    return 0;
}
```

Sample Output:

```
Enter rows and columns: 2 2
Enter elements of matrix A:
1 2
3 4
Enter elements of matrix B:
5 6
7 8
Sum matrix:
6 8
10 12
```

Q2. Write a program to read n number of values into an array and display them in reverse order.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr, n, i;
    printf("Enter n: ");
```

```

scanf("%d", &n);

arr = (int *)malloc(n * sizeof(int));

printf("Enter %d elements: ", n);
for (i = 0; i < n; i++)
    scanf("%d", &arr[i]);

printf("Reversed array: ");
for (i = n - 1; i >= 0; i--)
    printf("%d ", arr[i]);
printf("\n");

free(arr);
return 0;
}

```

Sample Output:

```

Enter n: 5
Enter 5 elements: 10 20 30 40 50
Reversed array: 50 40 30 20 10

```

Q3. Count the total number of duplicate elements in an array.

Code:

```

#include <stdio.h>
int main() {
    int arr[100], n, i, j, c = 0;

    printf("Enter n: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    for (i = 1; i < n; i++) {
        for (j = 0; j < i; j++) {
            if (arr[i] == arr[j]) {
                c++;
                break;
            }
        }
    }

    printf("Total duplicate elements: %d\n", c);

    return 0;
}

```

Sample Output:

```

Enter n: 6
Enter 6 elements: 1 2 2 3 3 3
Total duplicate elements: 3

```

Q4. Print the unique elements in an array.

Code:

```

#include <stdio.h>
int main() {
    int arr[100], n, i, j, c;

    printf("Enter n: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);

```

```

for (i = 0; i < n; i++)
    scanf("%d", &arr[i]);

printf("Unique elements: ");
for (i = 0; i < n; i++) {
    c = 0;
    for (j = 0; j < n; j++)
        if (arr[i] == arr[j])
            c++;
    if (c == 1)
        printf("%d ", arr[i]);
}

printf("\n");

return 0;
}

```

Sample Output:

```

Enter n: 6
Enter 6 elements: 1 2 2 3 3 3
Unique elements: 1

```

Q5. Insert an element into a sorted array.

Code:

```

#include <stdio.h>
int main() {
    int arr[100], n, i, key, pos;

    printf("Enter n: ");
    scanf("%d", &n);

    printf("Enter %d sorted elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter element to insert: ");
    scanf("%d", &key);

    pos = n;

    for (i = 0; i < n; i++) {
        if (arr[i] > key) {
            pos = i;
            break;
        }
    }

    for (i = n; i > pos; i--)
        arr[i] = arr[i - 1];
    arr[pos] = key;
    n++;

    printf("Array after insertion: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\n");

    return 0;
}

```

Sample Output:

```

Enter n: 5
Enter 5 sorted elements: 10 20 30 40 50

```

Enter element to insert: 25
Array after insertion: 10 20 25 30 40 50

Q6. Insert an element into an unsorted array.

Code:

```
#include <stdio.h>
int main() {
    int arr[100], n, i, key, pos;

    printf("Enter n: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter element to insert: ");
    scanf("%d", &key);

    printf("Enter position (0-indexed): ");
    scanf("%d", &pos);

    for (i = n; i > pos; i--)
        arr[i] = arr[i - 1];
    arr[pos] = key;
    n++;

    printf("Array after insertion: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\n");

    return 0;
}
```

Sample Output:

Enter n: 5
Enter 5 elements: 10 40 20 50 30
Enter element to insert: 99
Enter position (0-indexed): 2
Array after insertion: 10 40 99 20 50 30

Q7. Matrix multiplication.

Code:

```
#include <stdio.h>
int main() {
    int r1, c1, r2, c2, i, j, k;

    printf("Enter rows and columns of A: ");
    scanf("%d %d", &r1, &c1);

    printf("Enter rows and columns of B: ");
    scanf("%d %d", &r2, &c2);

    if (c1 != r2) {
        printf("Multiplication not possible\n");
        return 0;
    }

    int a[r1][c1], b[r2][c2], c[r1][c2];

    printf("Enter elements of A:\n");
```

```

for (i = 0; i < r1; i++)
    for (j = 0; j < c1; j++)
        scanf("%d", &a[i][j]);

printf("Enter elements of B:\n");
for (i = 0; i < r2; i++)
    for (j = 0; j < c2; j++)
        scanf("%d", &b[i][j]);

for (i = 0; i < r1; i++) {
    for (j = 0; j < c2; j++) {
        c[i][j] = 0;
        for (k = 0; k < c1; k++)
            c[i][j] += a[i][k] * b[k][j];
    }
}

printf("Product matrix:\n");
for (i = 0; i < r1; i++) {
    for (j = 0; j < c2; j++)
        printf("%d ", c[i][j]);
    printf("\n");
}

return 0;
}

```

Sample Output:

```

Enter rows and columns of A: 2 2
Enter rows and columns of B: 2 2
Enter elements of A:
1 2
3 4
Enter elements of B:
5 6
7 8
Product matrix:
19 22
43 50

```

Q8. Matrix transpose.

Code:

```

#include <stdio.h>
int main() {
    int r, c, i, j;

    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);

    int a[r][c], t[c][r];

    printf("Enter elements:\n");
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);

    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            t[j][i] = a[i][j];

    printf("Transpose:\n");
    for (i = 0; i < c; i++) {
        for (j = 0; j < r; j++)
            printf("%d ", t[i][j]);
        printf("\n");
    }
}

```

```

    }

    return 0;
}

```

Sample Output:

Enter rows and columns: 2 3

Enter elements:

1 2 3

4 5 6

Transpose:

1 4

2 5

3 6

Q9. Find the right diagonal sum of a square matrix.

Code:

```

#include <stdio.h>
int main() {
    int n, i, j, sum = 0;

    printf("Enter n (square matrix size): ");
    scanf("%d", &n);

    int a[n][n];

    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    for (i = 0; i < n; i++)
        sum += a[i][n - 1 - i];

    printf("Right diagonal sum: %d\n", sum);

    return 0;
}

```

Sample Output:

Enter n (square matrix size): 3

Enter elements:

1 2 3

4 5 6

7 8 9

Right diagonal sum: 15

Q10. Check whether a given square matrix is an identity matrix.

Code:

```

#include <stdio.h>
int main() {
    int n, i, j, flag= 1;

    printf("Enter n (square matrix size): ");
    scanf("%d", &n);

    int a[n][n];

    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);
    for (i = 0; i < n; i++) {

```

```

        for (j = 0; j < n; j++) {
            if (i == j && a[i][j] != 1) flag = 0;
            if (i != j && a[i][j] != 0) flag = 0;
        }

    if (flag)
        printf("Identity matrix\n");
    else
        printf("Not an identity matrix\n");

    return 0;
}

```

Sample Output:

```

Enter n (square matrix size): 3
Enter elements:
1 0 0
0 1 0
0 0 1
Identity matrix

```

Q11. Find the number of vowels, consonants, digits and white spaces in a sentence.

Code:

```

#include <stdio.h>
#include <stdlib.h>
int main() {
    char *str;
    int size = 1000;
    int i, v = 0, c = 0, d = 0, s = 0;
    char ch;

    str = (char *)malloc(size * sizeof(char));

    printf("Enter a sentence: ");
    fgets(str, size, stdin);

    for (i = 0; str[i] != '\0'; i++) {
        ch = str[i];
        if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z')) {
            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
                ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U')
                v++;
            else
                c++;
        } else if (ch >= '0' && ch <= '9') {
            d++;
        } else if (ch == ' ') {
            s++;
        }
    }

    printf("Vowels: %d\n", v);
    printf("Consonants: %d\n", c);
    printf("Digits: %d\n", d);
    printf("White spaces: %d\n", s);

    free(str);
    return 0;
}

```

Sample Output:

```

Enter a sentence: Hello World 123
Vowels: 3
Consonants: 7

```

Digits: 3
White spaces: 2

Q12. Find the frequency of a character in a string.

Code:

```
#include <stdio.h>
int main() {
    char str[100], ch;
    int i, count = 0;

    printf("Enter a string: ");
    fgets(str, 100, stdin);

    printf("Enter character to find: ");
    scanf("%c", &ch);

    for (i = 0; str[i] != '\0'; i++) {
        if (str[i] == ch)
            count++;
    }

    printf("Frequency of '%c': %d\n", ch, count);

    return 0;
}
```

Sample Output:

```
Enter a string: programming
Enter character to find: r
Frequency of 'r': 2
```

Q13. Count the number of words in a sentence.

Code:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    char *str;
    int size = 100;
    int i, count = 0;

    str = (char *)malloc(size * sizeof(char));

    printf("Enter a sentence: ");
    fgets(str, size, stdin);

    for (i = 0; str[i] != '\0'; i++) {
        if (str[i] == ' ' && str[i + 1] != ' ' && str[i + 1] != '\0' && str[i + 1] != '\n')
            count++;
    }
    if (str[0] != '\0' && str[0] != '\n')
        count++;

    printf("Number of words: %d\n", count);

    free(str);
    return 0;
}
```

Sample Output:

```
Enter a sentence: This is a sample sentence
Number of words: 5
```

Q14. Sort an array of strings.

Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    char arr[100][100], temp[100];
    int n, i, j;

    printf("Enter number of strings: ");
    scanf("%d", &n);

    printf("Enter %d strings:\n", n);
    for (i = 0; i < n; i++)
        scanf("%s", arr[i]);

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (strcmp(arr[j], arr[j + 1]) > 0) {
                strcpy(temp, arr[j]);
                strcpy(arr[j], arr[j + 1]);
                strcpy(arr[j + 1], temp);
            }
        }
    }
    printf("Sorted strings:\n");
    for (i = 0; i < n; i++)
        printf("%s\n", arr[i]);

    return 0;
}
```

Sample Output:

```
Enter number of strings: 5
Enter 5 strings:
banana apple mango cherry grape
Sorted strings:
apple
banana
cherry
grape
mango
```

Q15. Toggle the case of a sentence (lowercase to uppercase and vice versa).

Code:

```
#include <stdio.h>
int main() {
    char str[100];
    int i;

    printf("Enter a sentence: ");
    fgets(str, 100, stdin);

    for (i = 0; str[i] != '\0'; i++) {
        if (str[i] >= 'a' && str[i] <= 'z')
            str[i] = str[i] - 32;
        else if (str[i] >= 'A' && str[i] <= 'Z')
            str[i] = str[i] + 32;
    }

    printf("Toggled sentence: %s", str);

    return 0;
}
```

Sample Output:

Enter a sentence: Hello World

Toggled sentence: hELLO wORLD